UNIT-V

UNIT-V : **GRAPHICAL MODELS**

- Markov Chain Monte Carlo Methods (Book-1) –
  - Sampling
  - Proposal Distribution
  - Markov Chain Monte Carlo
  - Graphical Models
  - Bayesian Networks
  - Markov Random Fields
  - Hidden Markov Models
  - Tracking Methods

Markov Chain Monte Carlo (MCMC) Methods

- This method has revolutionized statistical computing and statistical physics over the past 20 years.
- This algorithm has been around since 1953, but only when computers became fast enough to be able to perform the computations on real-world examples in hours instead of weeks did the methods become really well known.
- However, this algorithm has now been cited as one of the most influential ever created.
- Two basic problems that can be solved using these methods, Compute the
  - Optimum solution to some objective function, or
  - Posterior distribution of a statistical learning problem.

- In either case the state space may well be very large, and we are only interested in finding the best possible answer .

- It means, what Monte Carlo sampling is, and look at Markov chains.

SAMPLING

- Produce samples from probability distributions

  Ex: Initialization of weights

- In many cases, the probability distribution used has been the uniform one on [0, 1), and done it using Gaussian distributions.

RANDOM NUMBERS

- The basis of all of these sampling methods is in the generation of random numbers, and computers are not really capable of doing.
- The simplest of the algorithms that produce pseudo-random numbers is the linear congruential generator.
- This is a very simple function and is defined by a recurrence relation

$$\mathbf{x_{n+1} = (ax_n + c) \bmod m \quad --- \quad (15.1)}$$

- Choose **a**, **c**, and **m** are parameters carefully.
  - m=modulus (m>0)
  - a=multiplier (0<a<m)
  - c=increment (0<=a<m)
  - $\mathbf{x_0}$ = starting value/seed value ((0<=$\mathbf{x_0}$ <m)
- All the parameters, initial input $\mathbf{x_0}$ (known as the seed), and the outputs are integers.

Gaussian Random Numbers

- The Mersenne twister produces uniform random numbers.
- Produce samples from other distributions, e.g., Gaussian.
- The usual method is the Box–Muller scheme, which uses a pair of uniformly randomly distributed numbers in order to make two independent Gaussian-distributed numbers with zero mean and unit variance.
- The product of two independent zero mean, unit variance normals is:

$$f(x,y) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2} \frac{1}{\sqrt{2\pi}} e^{-y^2/2} = \frac{1}{2\pi} e^{-(x^2+y^2)/2}. \qquad (15.2)$$

- If we use polar coordinates instead (so $x = r \sin(\Theta)$ *and* $y = r \cos(\Theta)$*) then we would* have $r^2 = x^2 + y^2$ *and* $\Theta = \tan{-}1(y/x)$.
- *Both of these are uniformly distributed random* variables ($0 \leq r \leq 1$ *and* $0 \leq \Theta < 2\prod$).
- $\Theta = 2 \prod U_1$ *where* U_1 *is a uniformly* distributed random variable.

- U_2 *is another uniformly* distributed random variable, and which has solution $r = \sqrt{-2 \ln U_2}$
- One algorithm to generate the Gaussian variables is:

The Box–Muller Scheme

- Pick two uniformly distributed random numbers $0 \leq U_1, U_2 \leq 1$
- Set $\theta = 2\pi U_1$ and $r = \sqrt{(-2\ln(U_2))}$
- Then $x = r\cos(\theta)$ and $y = r\sin(\theta)$ are independent Gaussian-distributed variables with zero mean and unit variance

• Other approach to compute random variables is to pick the two uniform random values and scale them to lie between -1 and 1, and to interpret them as a point in the plane.

• If this point is outside the unit circle ($w^2 = U_1^2 + U_2^2 > 1$) *then it is discarded, and another point picked until* it is within the circle.

• Then the transformation is

- Similarly for y *with* U_2 *also provides the variables.*
- The difference between the methods is that one requires the computation of sin(Θ), while the other requires that some points are sampled and discarded.
- Which is faster depends upon the programming language and computer architecture.
- A plot of 1,000 samples created by the Box–Muller scheme along with the zero mean, unit variance Gaussian line is shown in Figure 15.1.

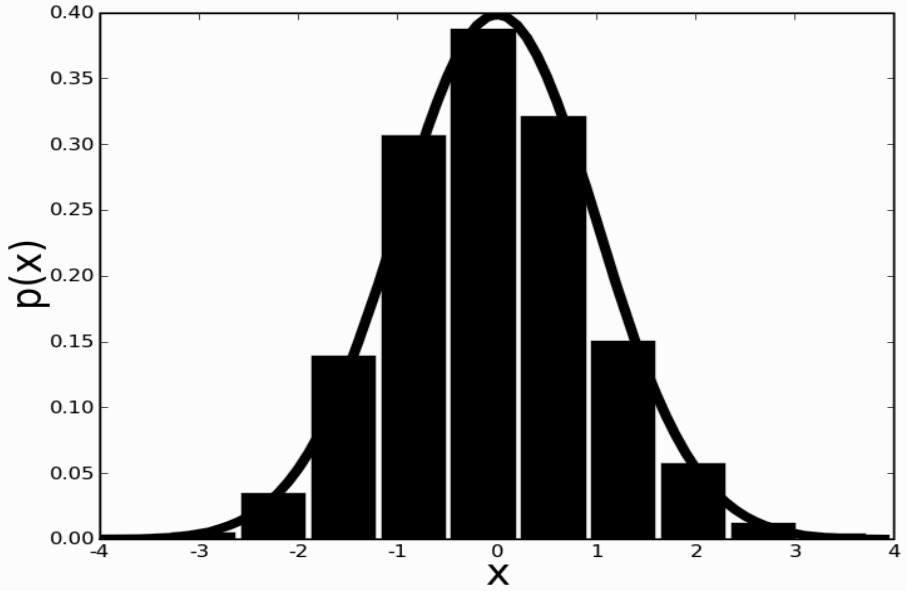


FIGURE 15.1 Histogram of 1,000 Gaussian samples created by the Box–Muller scheme. The line gives the Gaussian distribution with zero mean and unit variance.

MONTE CARLO OR BUST

- Monte Carlo, a tiny principality on the Mediterranean coast between France and Italy, is famous mostly for its casino and Grand Prix race.
- Rich and famous flock to lose money there, they are unlikely to know that the principality also has the dubious honour of having an important statistical principle named after it.
- The Monte Carlo principle states that if you take independent and identically distributed (i.e., well-behaved) samples $\mathbf{x}^{(i)}$ from an unknown high-dimensional distribution $\boldsymbol{p(x)}$, then as the number of samples gets larger the sample distribution will converge to the true distribution.

Dirac delta

$$p_N(\mathbf{x}) = \frac{1}{N}\sum_{i=1}^{N}\delta(\mathbf{x}^{(i)} = \mathbf{x})$$

$$\rightarrow \lim_{N\rightarrow\infty} p_N(\mathbf{x}) = p(\mathbf{x}), \qquad (15.5)$$

where $\delta(\mathbf{x}_i = \mathbf{x})$ is the Dirac delta function that is 0 everywhere except at the point $\mathbf{x}_i$ and has $\int \delta(x)dx = 1$. This can be used to compute the expectation as well (where $f(\mathbf{x})$ is some function and $\mathbf{x}$ has discrete values, and the superscript $.^{(i)}$ represents the index of the sample):

$$E_N(f) = \frac{1}{N}\sum_{i=1} f(\mathbf{x}^{(i)})$$

$$\rightarrow \lim_{N\rightarrow\infty} E_N(f) = \sum_{\mathbf{x}} f(\mathbf{x})p(\mathbf{x}). \qquad (15.6)$$

- The fact that the sample distribution becomes more and more true as samples are more likely to be drawn from parts of the distribution that have high probability.
- The whole of probability theory was originally developed by some of the great French mathematicians, such as **Fermat**, in order to reason about games of chance.
- Monte Carlo principle tells you that it is exactly what you should be doing.
- Suppose that you play ten **games of patience** (or **solitaire** is a genre of card **games**) and six of them come out i.e. approximately 60% of the patience games will do well.

THE PROPOSAL DISTRIBUTION

- Distribution p(x) that we are sampling from is easy (that is, not computationally expensive).
- Unfortunately, this is very rare case, but fortunately there is a way to get around this problem.

$$p(\mathbf{x}) = \frac{1}{Z_p}\tilde{p}(\mathbf{x}), \tag{15.8}$$

We make the decision of whether or not to accept the sample by picking a uniformly distributed random number u between 0 and $Mq(\mathbf{x}^*)$. If this random number is less than $\tilde{p}(\mathbf{x}^*)$, then we accept $\mathbf{x}^*$, otherwise we reject it. The reason why this works is known as the envelope principle: the pair $(\mathbf{x}^*, u)$ is uniformly distributed under $Mq(\mathbf{x}^*)$, and the rejection part throws away samples that don't match the uniform distribution on $p(\mathbf{x}^*)$, so $Mq(\mathbf{x})$ forms an envelope on $p(\mathbf{x})$. Figure 15.2 shows the idea: we sample from $Mq(\mathbf{x})$ and reject any sample that lies in the grey area. The smaller M is, the more samples we get to keep, but we need to ensure that $\tilde{p}(\mathbf{x}) \leq Mq(\mathbf{x})$. This method is known as rejection sampling, and the algorithm can be written as:

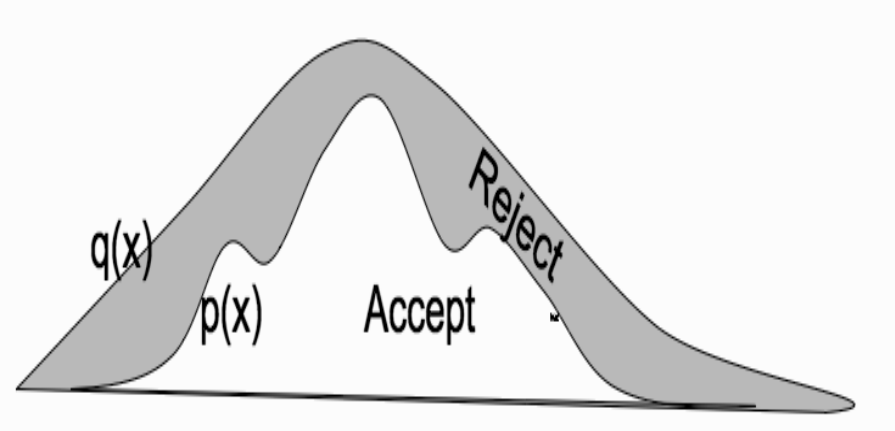


FIGURE 15.2 The proposal distribution method.

The Rejection Sampling Algorithm

- Sample $\mathbf{x}^*$ from $q(\mathbf{x})$ (e.g., using the Box–Muller scheme if $q(\mathbf{x})$ is Gaussian)
- Sample u from uniform$(0, \mathbf{x}^*)$
- If $u < p(\mathbf{x}^*)/Mq(\mathbf{x}^*)$:
  - add $\mathbf{x}^*$ to the set of samples
- Else:
  - reject $\mathbf{x}$ and pick another sample

- As an example of using rejection sampling, Figure 15.3 shows the results of using it to sample from the mixture of two Gaussians by using the uniform distribution shown by the dotted line.
- Using *M = 0.8, as shown in the fig., the algorithm rejects about half of* the samples. Using *M = 2 the algorithm rejects about 85% of samples.*

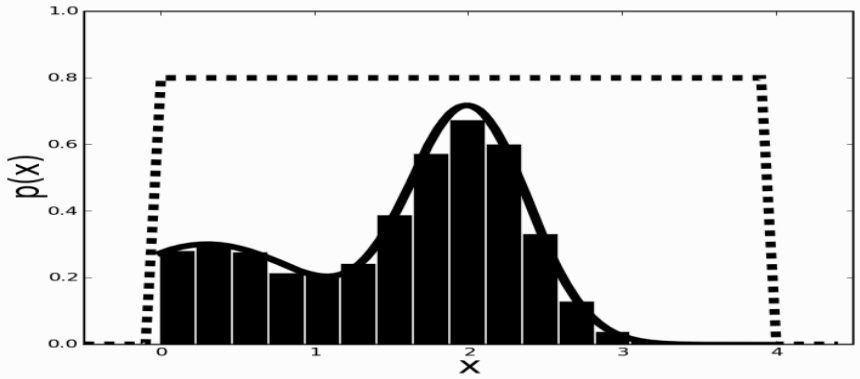


FIGURE 15.3 The histogram shows samples of a mixture of two Gaussians (given by the solid line) as sampled from the uniform box shown as a dotted line by using rejection sampling.

- So with rejection sampling, throw away samples, and if you don't pick *M properly, you will* have to reject a lot of them.
- The curse of dimensionality makes the problem even worse.
- There are two things that we can do to get over this problem.
- One is to develop more sophisticated methods of understanding the space that we are sampling, and the other is to try to ensure that the samples are taken from areas of the space that have high probability.

- A method that tries to ensure that the samples come from regions of high probability and is known as Importance Sampling, because it attaches a weight that says how important each sample is.
- To compute the expectation of a function f(x) for a c.r.v x distributed according to unknown distribution p(x).
- Starting from the expression of the expectation that we wrote out earlier, we can introduce another distribution q(x):

$$\begin{aligned} E(f) &= \int p(\mathbf{x}) f(x)\, d\mathbf{x} \\ &= \int p(\mathbf{x}) f(\mathbf{x}) \frac{q(\mathbf{x})}{q(\mathbf{x})} d\mathbf{x} \\ &\approx \frac{1}{N} \sum_{i=1}^{N} \frac{p\left(\mathbf{x}^{(i)}\right)}{q\left(\mathbf{x}^{(i)}\right)} f\left(x^{(i)}\right), \end{aligned} \tag{15.9}$$

where we have used the fact that $q(\mathbf{x})$ is the density of a random variable, and so if we perform $\int q(\mathbf{x}) d\mathbf{x}$ over all values of $\mathbf{x}$, then it must equal 1. The ratio $w(\mathbf{x}^{(i)}) = p(\mathbf{x}^{(i)})/q(\mathbf{x}^{(i)})$ is called the importance weight, and it corrects for sampling from the grey region in Figure 15.2 without having to reject samples. While this can be used to estimate the expectation directly, the real benefit of computing the importance weights is that they can be used in order to resample the data. This leads to an algorithm known descriptively as Sampling-Importance-Resampling. In the words of the advert, it 'does exactly what it says on the tin':

The Sampling-Importance-Resampling Algorithm

- Produce N samples $\mathbf{x}^{(i)}$, $i = 1 \ldots N$ from $q(\mathbf{x})$
- Compute normalised importance weights

$$w^{(i)} = \frac{p(\mathbf{x}^{(i)})/q(\mathbf{x}^{(i)})}{\sum_j p(\mathbf{x}^{(j)})/q(\mathbf{x}^{(j)})} \tag{15.10}$$

- Resample from the distribution $\{\mathbf{x}^{(i)}\}$ with probabilities given by the weights $w^{(i)}$

- The results of using sampling importance-Resampling on the example in Figure 15.3 are given in Figure 15.4.
- Note that this method does not reject any samples, but it does involve two separate sampling steps and a relatively expensive loop.
- Like the other algorithms we have seen, it is sensitive to the quality of the match between the proposal distribution *q(**x**) **and the actual distribution** p(**x**).*

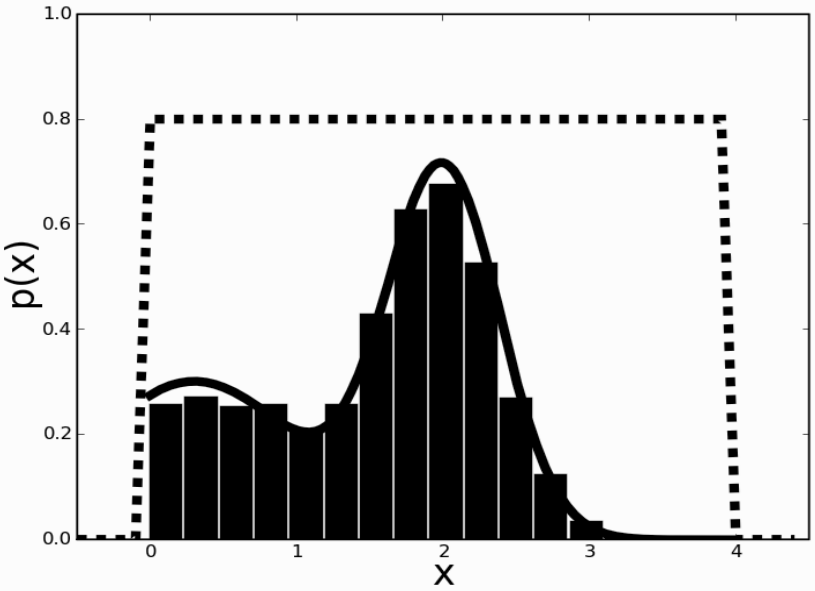


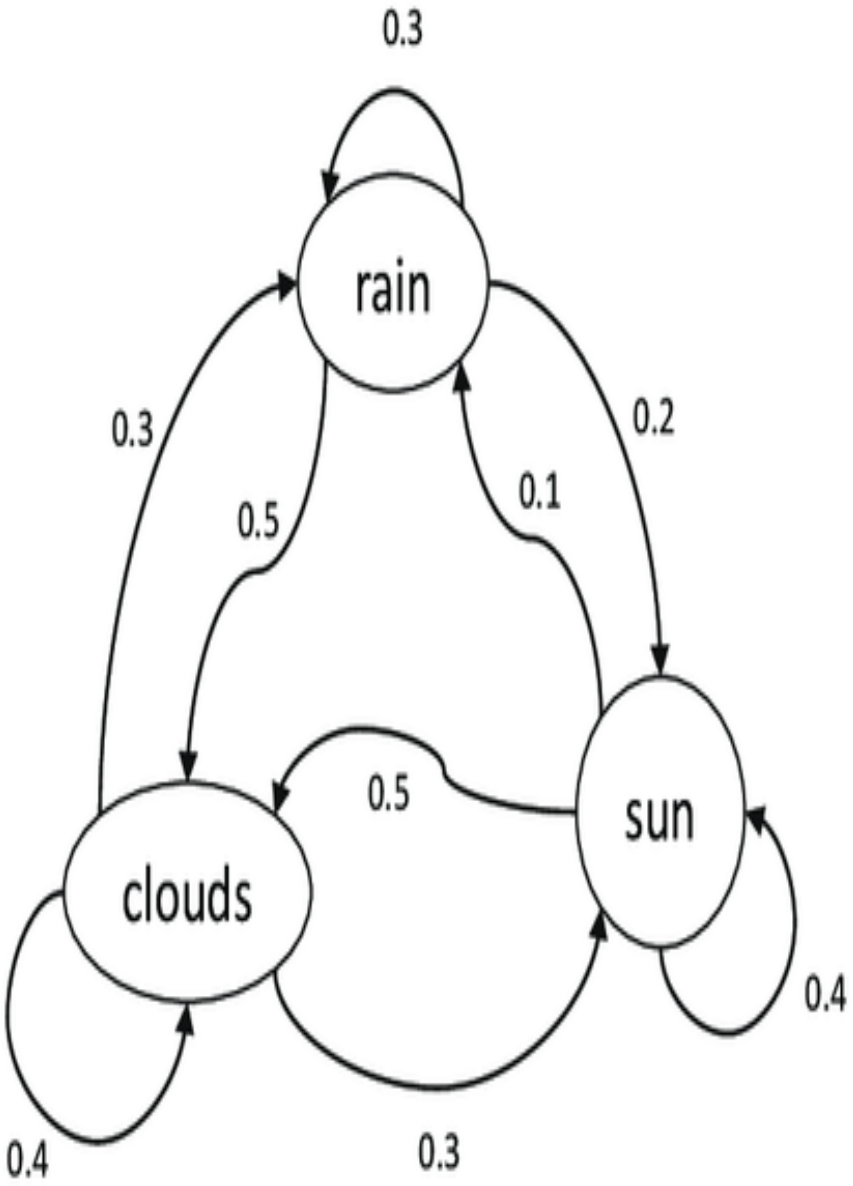
FIGURE 15.4 The histogram shows samples created using sampling-importance-Re sampling from a mixture of two Gaussians (given by the solid line) as sampled from the uniform box shown as a dotted line.

- A method that uses sampling-importance-Resampling in an on-line application, known as a particle filter or sequential Monte Carlo method.
- How to find out sample space.
- The basic idea is to keep track of the sequence of samples and modify the proposal distribution to take advantage of this, for which we will have to use some more complicated machinery.

MARKOV CHAIN MONTE CARLO

MARKOV CHAINS

- In probabilistic terms a chain is a sequence of possible states, where the probability of being in state *s at time t is a function of the previous states*.
- *A Markov chain is a chain with* the Markov property, i.e., the probability at time *t depends only on the state at t − 1*.
- The set of possible states are **linked together by transition probabilities** that say how likely it is that you move from the current state to each of the others, and they are generally written as a matrix *T*.
- *They might be constant, or functions* of some other variables, but here we will assume that they are constant.
- There is no action here that affects the probability of moving into a particular state.
- Given a chain, we can perform a random walk on the chain by choosing a start state and randomly choosing each successive state according to the transition probabilities.



	clouds	rain	sun
clouds	0.4	0.3	0.3
rain	0.5	0.3	0.2
sun	0.5	0.1	0.4

- The link to sampling that we need is that if the transition probabilities reflect the distribution that we wish to sample from, then a random walk will explore that distribution.
- One problem with this is that random walks are very inefficient at exploring space, since they move back towards the start as often as they move away, which means the distance they move from the start scales as $\sqrt{t}$, *where t is the number of samples.*
- *Need to explore more* efficiently than just using a random walk.
- Do this by setting up our Markov chain so that it reflects the distribution we wish to sample from, and we want the distribution $p(x^{(i)})$ to converge to the actual distribution $p(x)$ no matter what state we start from.
- Since we can start from any state, this tells us that every state is reachable from every other state, which means that the chain is irreducible so that the transition matrix can't be cut up into smaller matrices.

- The chain also has to be **ergodic,** which means that we will revisit every state, so that the probability of visiting any particular state in the future never goes to zero, but is **not periodic**, which means that we can visit at any time, not just every ***k*** *iterations for some constant k.*
- The distribution p(x) to be invariant to the Markov chain, which means that the transition probabilities don't change the distribution:

$$p(\mathbf{x}) = \sum_{\mathbf{y}} T(\mathbf{y}, \mathbf{x}) p(\mathbf{y}). \qquad (15.11)$$

- Finding the transition probabilities to make this true: Move backwards and forwards along the chain with equal probability, so that the chain is reversible.
- Probability of being in an unlikely state *s (sampling data point* x), but heading for a likely state s^i *(data point* x^i) should be the same as being in the likely state s^i *and heading for the unlikely state s, so that:*

$$p(\mathbf{x})T(\mathbf{x}, \mathbf{x}') = p(\mathbf{x}')T(\mathbf{x}', \mathbf{x}). \qquad (15.12)$$

- This is known as the detailed balance condition and the fact that it leaves the distribution p(x) alone is fairly obvious with a little calculation.
- If the chain satisfies the detailed balance condition, then it must be ergodic, $\sum_y T(x,y) = 1$ since you must have come from some state, and so

$$\sum_{\mathbf{y}} p(\mathbf{y})T(\mathbf{y},\mathbf{x}) = p(\mathbf{x}), \qquad (15.13)$$

- which means that p(x) must be an invariant distribution of T.
- So if we can work out how to construct a Markov chain with detailed balance we can sample from it in order to sample from our distribution.
- This is known as Markov Chain Monte Carlo (MCMC) sampling, and the most popular algorithm that is used for MCMC is the Metropolis–Hastings algorithm after the two people who were directly involved in its creation.

- The Metropolis–Hastings Algorithm We assume that we have a proposal distribution of the form $q(x^{(i)}|x^{(i-1)})$ that we can sample from.
- The **idea of Metropolis–Hastings is similar to that of rejection sampling:** we take a sample x and choose whether or not to keep it.
- Except, unlike rejection sampling**,** rather than picking another sample if we reject the current one, instead we add another copy of the previous accepted sample.
- Here, the probability of keeping the sample is $u(x^*|x^{(i-1)})$:

$$u(\mathbf{x}^*|\mathbf{x}^{(i)}) = \min\left(1, \frac{\tilde{p}(\mathbf{x}^*)q(\mathbf{x}^{(i)}|\mathbf{x}^*)}{\tilde{p}(\mathbf{x}^{(i)})q(\mathbf{x}^*|\mathbf{x}^{(i)})}\right). \tag{15.14}$$

- Each step involves using the current value to sample from the proposal distribution.
- These values are accepted if they move the Markov chain towards more likely states, and because the Markov chain is reversible (since it satisfies the detailed balance condition) the algorithm explores states that are proportional to the difficult distribution *p(x)*.

The Metropolis–Hastings Algorithm

- Given an initial value x_0
- Repeat
  - sample $\mathbf{x}^*$ from $q(\mathbf{x}_i|\mathbf{x}_{i-1})$
  - sample u from the uniform distribution
  - if $u <$ Equation (15.14):
    - set $\mathbf{x}[i+1] = \mathbf{x}^*$
  - otherwise:
    - set $\mathbf{x}[i+1] = \mathbf{x}[i]$
- Until you have enough samples

- The Metropolis–Hastings (and variants of it) are by far the most commonly used MCMC methods, and it is also the most general.
- It requires that you choose the proposal distribution q(x*|x) carefully, but it is a very simple algorithm to use.
- Figure 15.5 shows 5,000 samples computed using the algorithm on a mixture of two Gaussians based on a proposal distribution that is a single Gaussian.

FIGURE 15.5 The results of the Metropolis–Hastings algorithm when the true distribution is a mixture of two Gaussians (shown by the solid line) and the proposal distribution is a single Gaussian (the dotted line).

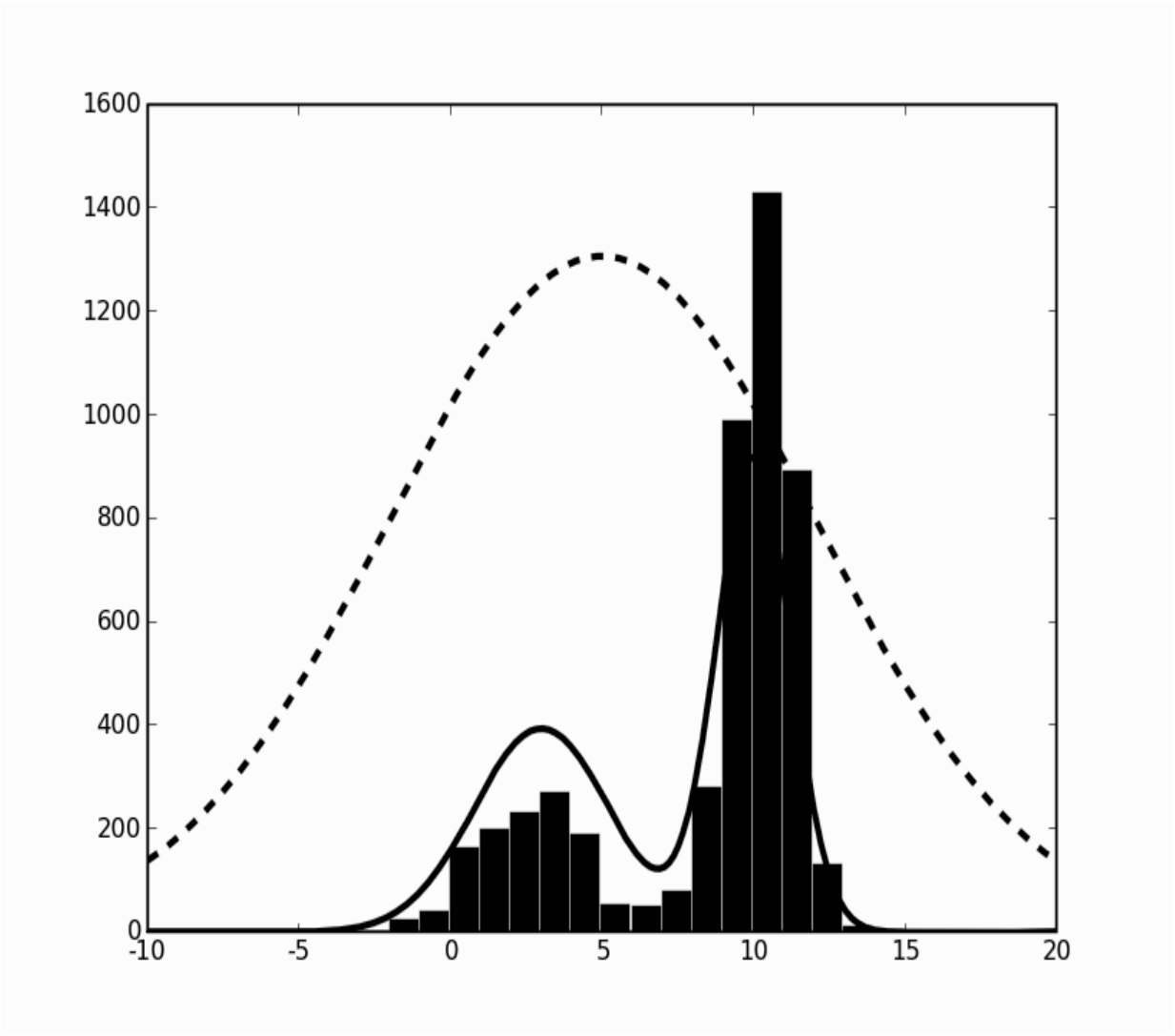
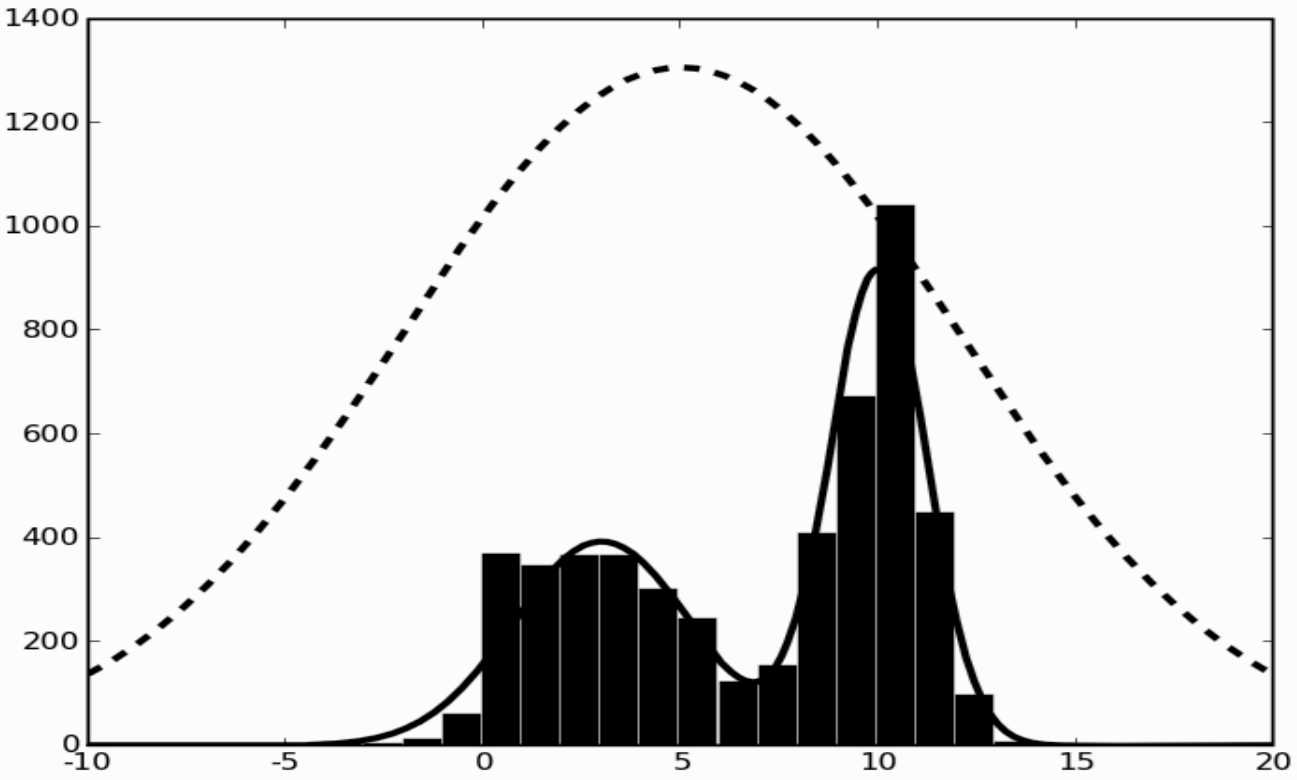


FIGURE 15.6 The results of the Metropolis algorithm when the true distribution is a mixture of two Gaussians (shown by the solid line) and the proposal distribution is a single Gaussian (the dotted line).



GRAPHICAL MODELS

- Machine Learning brings together computer science and statistics.
- Graphical models (or more completely, probabilistic graphical models), use graph theory with all its underlying computational and mathematical machinery in order to explain probabilistic models.
- The graphs used in graphical models are the exact ones that are taught in basic algorithms classes: a set of nodes, together with links between them, which can be either directed or not.

- There are two basic types of graphical models, depending upon whether or not the edges are directed.
- We will focus primarily on directed graphs, but the undirected kind (known as Markov Random Fields).
- For such a simple data structure, graphs have turned out to be incredibly powerful in many different parts of computer science, from constructing compilers to managing computer networks.
- For this reason, there are lots of readily available algorithms for finding shortest paths (Floyd's and Djiksta's algorithms), determining cycles, etc.
- Generate a node for each random variable, and label it accordingly.

- Only consider discrete variables, so that there is a finite number of possible values that the random variable can take.
- Given a continuous variable we will discretized it into a finite set.
- While this loses information, it makes the problem much simpler.
- The alternative is to specify the variable by a probability density function, which can be done, but makes the whole thing harder to describe and understand.
- If there is no connection between those two variables, which is the same as saying that they are independent.
- Except it isn't quite as simple as that, because two nodes could be linked through a third node.
- Have a look at the right of Figure 16.1, where C is not directly linked to B, but there is a link through A.
- For this reason we have to be careful and talk about conditional independence: C is conditionally independent of B, given A.

FIGURE 16.1 Two simple graphical models. The arrows denote causal relationships between nodes that represent features.

- Use directed links because these relationships are not symmetrical.
- Top of Figure 16.1, mean? ‘A’ causes ‘B’ (but note that this isn’t quite the same semantic usage that we normally have for ‘causes’, since there may be several variables that are all involved in causing B).
- Graph tells us that the probability of A and B is the same as the probability of A times the probability of B conditioned on A:

$$P(a, b) = P(b \mid a)\, P(a).$$

- If there is no direct link between two nodes then they are conditionally independent of each other.
- Third thing to specify the problem properly, is the conditional probability table for each variable.

- This specifies what the probabilities are for each of the nodes, conditioned on any nodes that are its parents.
- To work out a value for *P(a, b), it needs a distribution table for P(a) and one for P(b |a).*
- *The nodes are separated into those and their values are* directly as observed nodes—and hidden or latent nodes, whose values we hope to infer, and which may not have clear meanings in all cases.
- The basic concept of the graphical model is very simple, and it produces a powerful set of tools for understanding and creating machine learning algorithms.

BAYESIAN NETWORKS

- Consider directed graphs, and make one restriction to them, that they must not contain cycles, i.e. there cannot be any loops in the graphs.
- These graphs are Directed Acyclic graphs (DAGs), but for graphical models, when they are paired with the conditional probability tables, they are called Bayesian networks.
- Ex: Exam Fear
- Figure 16.2 shows a graph with a full set of distribution tables specified. It is a handy guide to whether or not you will be scared before an exam based on whether or not the course was boring (‘B’), which was the key factor you used to decide whether or not to attend lectures (‘A’) and revise (‘R’).
- To perform inference in order to decide the likelihood of you being scared before the exam (‘S’).
- Two kinds of inferences, depending on whether the observations that are made come from the top of the graph or the bottom.

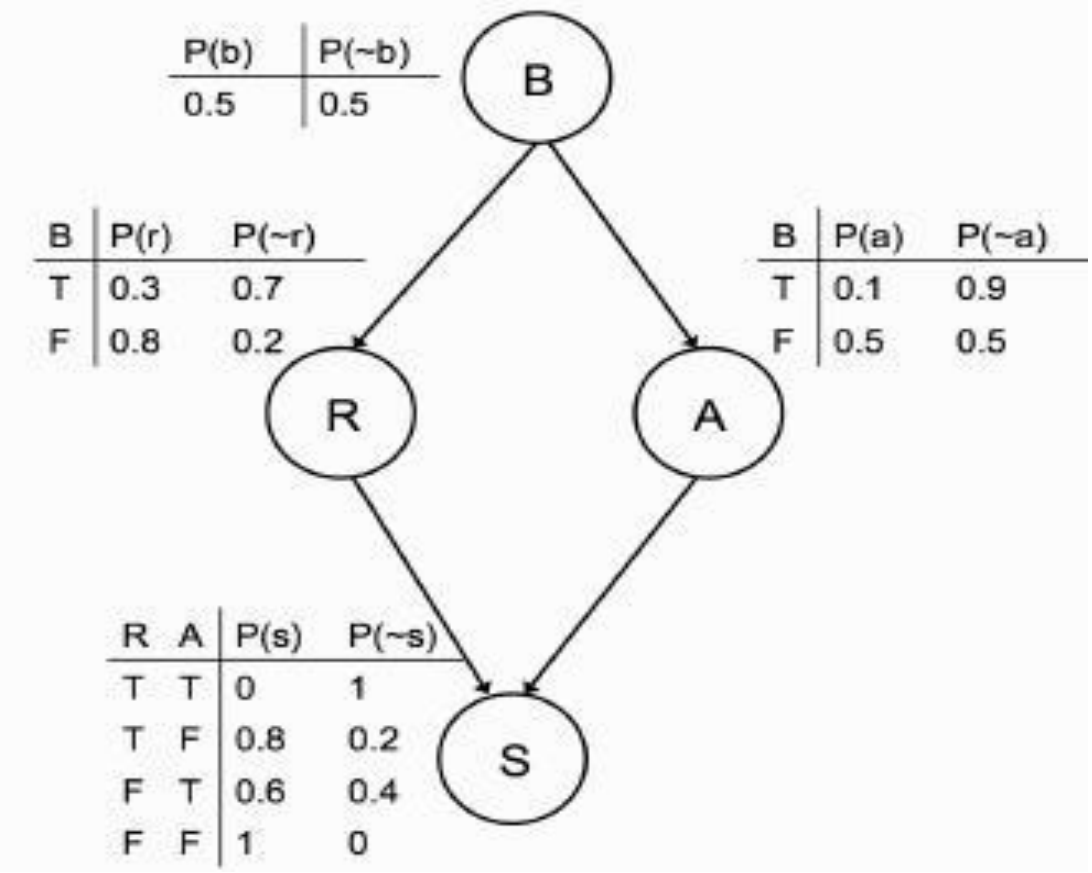


FIGURE 16.2 The sample graphical model. 'B' denotes a node stating whether the exam was boring, 'R' whether or not you revised, 'A' whether or not you attended lectures, and 'S' whether or not you will be scared before the exam.

- If we have a set of observations that can be used to predict an unknown outcome, then we are doing top-down inference or prediction, whereas if the outcome is known, but the causes are hidden, then we are doing bottom-up inference or diagnosis.

- To compute the probability of being scared, need to compute *P(b, r, a, s),* where the lower-case letters indicate particular values that the upper-case variables can take.
- Read the conditional probabilities from the graph—if there is no direct link, then variables are conditionally independent given a node that is already included, so those variables are not needed.

$$
\begin{aligned}
P(s) &= \sum_{b,r,a} P(b,r,a,s) \\
&= \sum_{b,r,a} P(b) \times P(r|b) \times P(a|b) \times P(s|r,a) \\
&= \sum_{b} P(b) \times \sum_{r,a} P(r|b) \times P(a|b) \times P(s|r,a). \qquad (16.1)
\end{aligned}
$$

- The conditional independence gives us even more: if I know both whether or not you attended lectures and whether or not you revised, then I don't need to know if the course was boring, since there is no direct connection between 'B' and 'S'.
- The probability of you being scared can be read off the final distribution table as 0.8.
- The power of the graphical model is when you don't have full information.
- Suppose you know that the course was boring, and want to work out how likely it is that you will be scared before the exam.
- In that case you can ignore the *P(b) terms, and just need to sum up the probabilities for r* and *a using Equation (16.1):*
- $P(s) = 0.3 \times 0.1 \times 0 + 0.3 \times 0.9 \times 0.8 + 0.7 \times 0.1 \times 0.6 + 0.7 \times 0.9 \times 1$

  $= 0.328$ ----- *(16.2)*

- The backwards inference, or diagnosis, can also be useful.
- Suppose that I see you looking very scared outside the exam.
- You look vaguely familiar, but I'm not sure whether or not you came to the lectures.
- Computations that need are (*P(s) is the* normalizing constant found by summing over all values of *r, a, and b, i.e., Equ. (16.2)):*

$$
\begin{aligned}
P(r|s) &= \frac{P(s|r)P(r)}{P(s)} \\
&= \frac{\sum_{b,a} P(b,a,r,s)}{P(s)} \\
&= \frac{0.5 \cdot (0.3 \cdot 0.1 \cdot 0 + 0.3 \cdot 0.9 \cdot 0.8) + 0.5 \cdot (0.8 \cdot 0.5 \cdot 0 + 0.8 \cdot 0.5 \cdot 0.8)}{P(s)} \\
&= \frac{0.268}{0.684} = 0.3918. \qquad (16.3)
\end{aligned}
$$

$$P(a|s) = \frac{P(s|a)P(a)}{P(s)}$$
$$= \frac{0.144}{0.684} = 0.2105. \qquad (16.4)$$

- This use of Bayes' rule is the reason why this type of graphical model is known as a Bayesian network.
- Even in this very simple example, the inference was not trivial, since there were a lot of calculations to do and the problem is actually rather worse than that.
- The computational cost of the simple algorithm we used (start at the root, and follow each link through the graph to perform the computation) is $O(2^N)$ *for a graph with* N nodes where each node can be either true or false.
- Problem of exact inference on Bayesian networks is NP-hard.

- it is rare to find such polytrees in real examples, so we can either try to turn other networks into polytrees, or consider only approximate inference, which is the most common solution to the problem.
- Speed up the things a little by getting things into the form of Equation (16.1), where the summations were carefully placed as far to the right as possible, so that program loops can be minimized.
- By doing this the algorithm is as efficient as possible, but it is still NP-hard.
- Idea is to convert the conditional probability tables into what are called *λ tables, which simply list all of the possible values for all variables, and* which initially contain the conditional probabilities. For Ex: the *λ table for the 'S'* variable in Figure 16.2 is:

R	A	S	λ
T	T	T	0
T	T	F	1
T	F	T	0.8
T	F	F	0.2
F	T	T	0.6
F	T	F	0.4
F	F	T	1
F	F	F	0

If I see you looking scared outside the exam (so that S is true), then I can eliminate it from the graph by removing from each table all rows that have S false in them, and deleting the S column. This simplifies things a little, but I have to do rather more in order to compute the probability of you having attended lectures.

- Don't know whether you revised or not, and don't know if you found the lectures boring, so marginalize over these variables.
- The order in which we marginalize doesn't change the correctness so pick R first.
- To eliminate R from the graph, we have to find all of the *tables that contain* it (there will be two of them containing R: the one for R itself and the one that we have just modified to remove S).

- To complete the entry where B is true and A is false, we have to multiply together the values where B, A, R are respectively true, false, true in the two tables and then add to that the product of where B, A, R are respectively true, false, false.
- In other words:

$$\left(\begin{array}{cc|c} B & R & \lambda \\ T & T & 0.3 \\ T & F & 0.7 \\ F & T & 0.8 \\ F & F & 0.2 \end{array}\right) \times \left(\begin{array}{cc|c} R & A & \lambda \\ T & T & 0 \\ T & F & 0.8 \\ F & T & 0.6 \\ F & F & 1 \end{array}\right) \Rightarrow \left(\begin{array}{cc|c} B & A & \lambda \\ T & T & 0.3 \cdot 0 + 0.7 \cdot 0.6 = 0.42 \\ T & F & 0.3 \cdot 0.8 + 0.7 \cdot 1 = 0.94 \\ F & T & 0.8 \cdot 0 + 0.2 \cdot 0.6 = 0.12 \\ F & F & 0.8 \cdot 0.8 + 0.2 \cdot 1 = 0.84 \end{array}\right) \quad (16.5)$$

• To see that these algorithms do not scale well, consider Figure 16.3, which shows a very simple development of the example in Figure 16.2 by adding just one extra node to the network: whether or not this is your final year ('F').
• This makes the network significantly more complicated, since we need another table and extra entries in two of the other tables, and therefore the variable elimination algorithm will take rather longer to run.

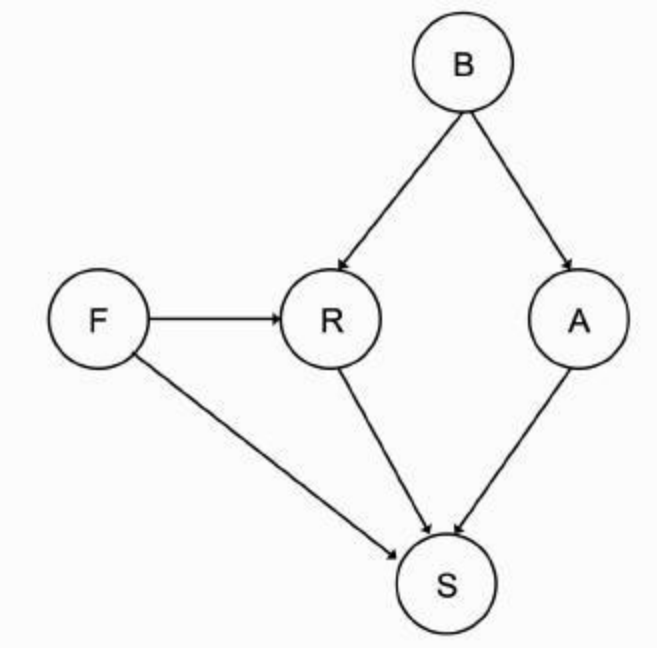


FIGURE 16.3 Adding just one extra node ('F', information about whether or not this is your final year) makes the conditional probability tables significantly more complicated.

The Variable Elimination Algorithm

- Create the λ tables:
  - for each variable v:
    - make a new table
    - for all possible true assignments x of the parent variables:
      - add rows for $P(v|x)$ and $1 - P(v|x)$ to the table
    - add this table to the set of tables
- Eliminate known variables v:
  - for each table:
    - remove rows where v is incorrect
    - remove column for v from table
- Eliminate other variables (where x is the variable to keep):
  - for each variable v to be eliminated:
    - create a new table t'
    - for each table t containing v:
      - $v_{\mathbf{true},t} = v_{\mathbf{true},t} \times P(v|x)$
      - $v_{\mathbf{false},t} = v_{\mathbf{false},t} \times P(\neg v|x)$

    - $v_{\text{true},t'} = \sum_t (v_{\text{true},t})$
    - $v_{\text{false},t'} = \sum_t (v_{\text{false},t})$
  - replace tables t with the new one t'

- Calculate conditional probability:
  - for each table:
    - $x_{\text{true}} = x_{\text{true}} \times P(x)$
    - $x_{\text{false}} = x_{\text{false}} \times P(\neg x)$
    - probability is $x_{\text{true}}/(x_{\text{true}} + x_{\text{false}})$

- The basic idea of using MCMC methods in Bayesian networks is to sample from the hidden variables, and then weight the samples by their likelihoods.
- Creating the samples is very easy: for prediction, we start at the top of the graph and sample from each of the known probability distributions.
- Sample from each distribution independently, and then throw away any samples that don't match the other variables.
- Which is computationally easier, but need to reject a lot of samples.
- Gibbs sampling will find us the maxima of our probability distribution given enough samples.
- For any node consider its parents, its children, and the other parents of the children, as shown in Figure 16.4. and this set is known as the **Markov blanket of a node**.

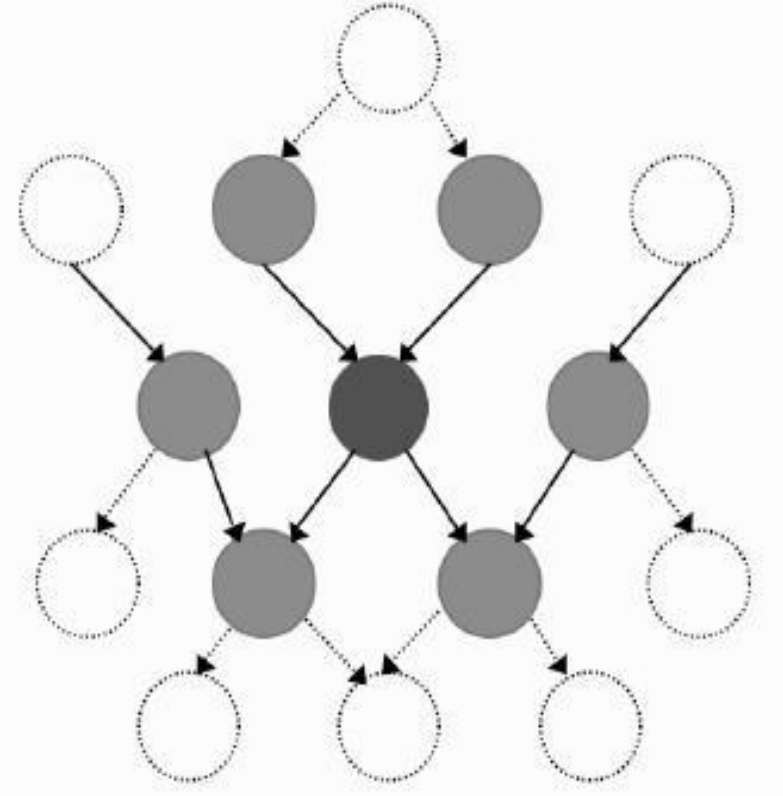


FIGURE 16.4 The Markov blanket of a node is the set of nodes (shaded light grey) that are either parents or children of the node, or other parents of its children (shaded dark grey).

Making Bayesian Networks

- If we are given the structure and conditional probability tables of the Bayesian network, then we can perform inference on it by using Gibbs sampling or, if the network is simple enough, exactly.

Case study (Naive Bayes)

- It is supervised learning alg
- It is one of the most fast and easy ML alg to predict a class of datasets
- Used to solve the classification problems (Binary /Multi-class)
- It is one of the simplest and effective classification alg
- Popular for text classification problems
- It is also called probabilistic classification algorithm because will predict the output based on the probability
- Ex:to filter the spam mails, sentiment analysis, article classification etc…
- Why it is called as Naive Bayesian classification

  Naive: If we have independent variable, based on the independent variable properties will do classification

  ex : Fruit : properties color :red, shape : round , Taste:sweet → apple

  Bayes: Based on the bayesian theorem principle it will work

Whether it is possible to play in sunny or not?

	Outlook	Play
0	Rainy	Yes
1	Sunny	Yes
2	Overcast	Yes
3	Overcast	Yes
4	Sunny	No
5	Rainy	Yes
6	Sunny	Yes
7	Overcast	Yes
8	Rainy	No
9	Sunny	No
10	Sunny	Yes
11	Rainy	No
12	Overcast	Yes
13	Overcast	Yes

Working of Naïve Bayes' Classifier:

1. Convert the given dataset into frequency tables.
2. Generate Likelihood table by finding the probabilities of given features.
3. Now, use Bayes theorem to calculate the posterior probability.

Frequency table for the Weather Conditions:

Weather	Yes	No
Overcast	5	0
Rainy	2	2
Sunny	3	2
Total	10	5

Likelihood table weather condition:

Weather	No	Yes	
Overcast	0	5	5/14= 0.35
Rainy	2	2	4/14=0.29
Sunny	2	3	5/14=0.35
All	4/14=0.29	10/14=0.71	

Applying Bayes'theorem:

P(Yes|Sunny)= P(Sunny|Yes)*P(Yes)/P(Sunny)

P(Sunny|Yes)= 3/10= 0.3

P(Sunny)= 0.35

P(Yes)=0.71

So P(Yes|Sunny) = 0.3*0.71/0.35= **0.60**

P(No|Sunny)= P(Sunny|No)*P(No)/P(Sunny)

P(Sunny|NO)= 2/4=0.5

P(No)= 0.29

P(Sunny)= 0.35

So P(No|Sunny)= 0.5*0.29/0.35 = **0.41**

So as we can see from the above calculation that **P(Yes|Sunny)>P(No|Sunny)**

Hence on a Sunny day, Player can play the game.

MARKOV RANDOM FIELDS

- Bayesian networks are inherently asymmetric, since each edge had an arrow on it.
- If we remove this constraint, then there is no longer any idea of children and parent nodes.
- It also makes the idea of conditional independence that we saw for the Bayesian network easier: two nodes in a Markov Random Field (MRF) are conditionally independent of each other, given a third node, if there is no path between the two nodes that doesn't pass through the third node.
- This is actually a variation on the Markov property.
- The state of a particular node is a function only of the states of its immediate neighbours, since all other nodes are conditionally independent given its neighbours.

- There are particular applications where MRF methods have turned out to be particularly useful.

  Ex: Image De-Noising, when we talked about Auto-Associative Learning in the MLP.

- Suppose we have a Binary Image I *with pixel values* $I_{xi,\ xj} \in \{-1, 1\}$.

- This image is a representation of an 'Ideal' Image $I'_{xi,xj}$ *that has no noise in it, which is what we want to recover.*

- *If we assume* that the amount of noise is small, then there should be a good correlation between the values of each pixel in the two images, so $I_{xi,xj}$ *and* $I'_{xi,xj}$ *should be correlated.*

- Assume that within a small 'patch' or region in an image, there is good correlation between pixels (so $I_{xi,xj}$ *should correlate well with* $I_{xi+1,xj}$ *and its other neighbouring pixels (*$I_{xi,xj-1}$*, etc.).*

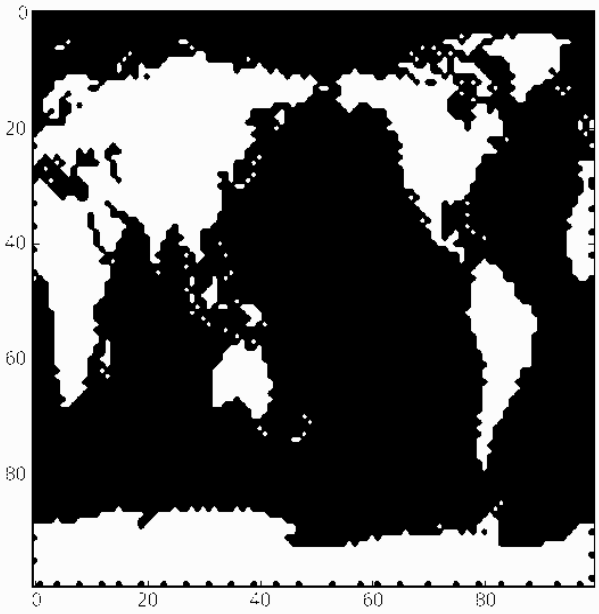
- Assumption says that there are lots of places in the image where all of the pixels are of the same value, and this is true for most images, and pixels are correlated (other pixels in the image are conditionally independent of $I_{xi,xj}$ given the neighbours of that pixel, which is the MRF bit)
- The original theory of MRFs was worked out by physicists, initially by looking at the Ising model, which is a statistical description of a set of atoms connected in a chain, where each can spin up (+1) or down (-1) and whose spin affects those connected to it in the chain.
- Write the energy of the same pixel in two images as $-\eta\ I_{xi,xj}$ $I'_{xi,xj}$ where η is a positive constant.

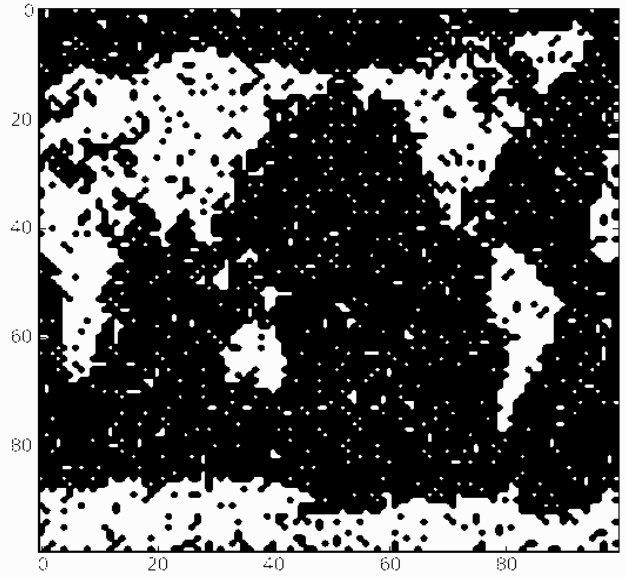
- If the two pixels have the same sign then the energy is negative, while if they have opposite signs then the energy is positive and therefore larger.
- The energy of two neighbouring pixels is $-\zeta I_{xi,xj} I_{xi+1,xj}$, *and* add these components together to get the total energy:

$$E(I, I') = -\zeta \sum_{i,j}^{N} I_{x_i,x_j} I_{x_i \pm 1, x_j \pm 1} - \eta \sum_{i,j=1}^{N} I_{x_i,x_j} I'_{x_i,x_j}, \qquad (16.9)$$

- Index of the pixels is assumed to run from 1 to *N in both the x and y directions in* both images and we are only interested in locally flat patches of the image we are changing, which is I.
- There is a simple iterative update algorithm, which is to start with noisy image I and ideal I', and update I so that at each step the energy calculation is lower.

- Pick one pixel $I_{xi,xj}$ for some values of xi, xj at a time, and compute the energies with this pixel being set to -1 and 1, picking the lower one.
- Make the probability of P(I, I') higher.
- The algorithm then moves on to another pixel, either choosing a random pixel at each step or moving through them in some pre-determined order, running through the set of pixels until their values stop changing.
- Figure 16.5 shows an original black and white image, a version corrupted with 10% noise, and the MRF-reconstructed version using parameters $\eta = 2.0, \zeta = 1.5$.
- This reduces the error from 10% to less than 1%, although it also removes New Zealand from the map!





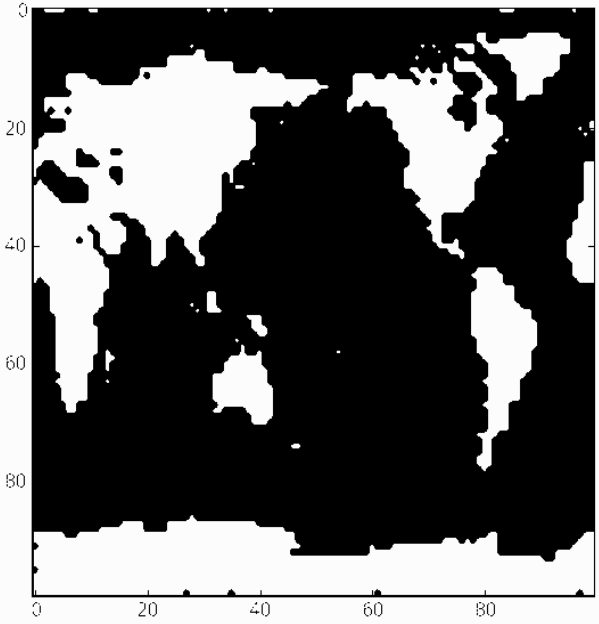


FIGURE 16.5 Using the MRF image de-noising algorithm with $\eta = 2.1$, $\zeta = 1.5$ on a map of the world (top right) corrupted by 10% uniformly distributed random noise (below left) gives the image which has about 1% error, although it has smoothed out the edges of all the continents.

The Markov Random Field Image Denoising Algorithm

- Given a noisy image I and an original image I', together with parameters η, ζ:
- Loop over the pixels of image I:
  - compute the energies with the current pixel being -1 and 1
  - pick the one with lower energy and set its value in I accordingly

- We will now focus on a type of graphical model that is in very common use, and that has computationally tractable algorithms for doing exact inference on it.

HIDDEN MARKOV MODELS (HMMS)

- Hidden Markov Model is one of the most popular graphical models.
- It is used in speech processing and in a lot of statistical work and works on a set of temporal data.
- At each clock tick the system moves into a new state, which can be the same as the previous one.
- Its power comes from the fact that it deals with situations where you have a Markov model, but you do not know exactly which state of the Markov model you are in—instead, you see observations that do not uniquely identify the state.
- This is where the ***hidden*** *in the title comes from.*
- *Performing inference on the HMM is not that computationally* expensive, which is a big improvement over the more general Bayesian network.

- The applications that most commonly applied are temporal: a set of measurements made at regular time intervals, which comprise the observations of the state.
- HMM is the simplest Dynamic Bayesian Network, a Bayesian Network deals with sequential (time-series) data.
- Figure 16.6 shows the HMM as a graphical model.
- Ex: As a caring teacher I want to know whether or not you are actually working towards the exam.
- There are four things that you do in the evenings (go to the pub, watch TV, go to a party, study) and I want to work out whether or not you are studying.
- However, I can't just ask you, because you would probably lie to me.

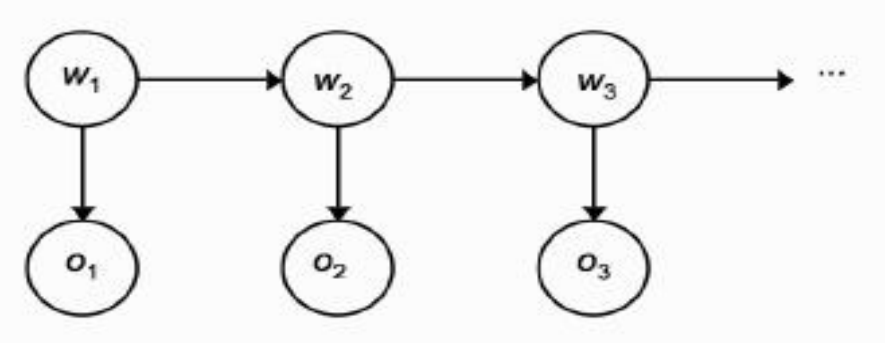


FIGURE 16.6 The Hidden Markov Model is an example of a Dynamic Bayesian network. The figure shows the first three states and the related observations unrolled as time progresses.

- Try to make observations about your behaviour and appearance.
- I can probably work out if you look tired, hungover, scared, or fine.
- Use these observations to try to work out what you did last night.
- The problem is that I don't know why you look the way you do, but I can guess by assigning probabilities to those things.
- So if you look hungover, then I might give probability 0.5 to the guess that you went to the pub last night, 0.25 to the guess that you went to a party, 0.2 to watching TV, and 0.05 to studying.
- Each day I see you in lectures I make an observation of your appearance, $o(t)$, *and* I want to use that observation to guess the state $\omega(t)$.

- Requires me to build up some kind of probabilities $P(o_k(t)|\ \omega_j(t))$, which is the probability that I see observation o_k (e.g., you are tired) given that you were in state ω_j (e.g., you went to a party) last night.
- These are usually labelled as $b_j(o_k)$.
- The other information that I have, or think I have, is the Transition Probability, which tells me how likely you are to be in state ω_j *tonight given that* you were in state ω_i *last night.*
- *So if I think you were at the pub last night I will probably* guess that the probability of you being there again tonight is small because your student loan won't be able to handle it.
- This is written as $P(\omega_j(t+1)|\ \omega_i(t))$ *and is usually labelled* as $a_{i,j}$.

- Know that you did something last night, so $\Sigma_j \alpha_{ij} = 1$ *and I know that I will make* some P observation (since if you aren't in the lecture I'll assume you were too tired), so $\Sigma_k b_j(o_k) = 1$.
- *There is one other thing that is generally assumed, which is that the Markov* chain is ergodic, it means that there is a non-zero probability of reaching every state eventually, no matter what the starting state.
- After a couple of weeks of the course I have made observations about you, and I am ready to sort out my HMM.
- There are three things that I might want to do with the data:
  - see how well the sequence of observations that I've made match my current HMM
  - work out the most probable sequence of states that you've been in based on my observations
  - given several sets of observations (for example, by watching several students) generate a good HMM for the data

- The HMM itself is made up of the transition probabilities $a_{i,j}$ *and the observation* probabilities $b_j(o_k)$*, and the probability of starting in each of the states,* Π_I*.*
- *So these are* the things need to be specified, starting with the transition probabilities shown in Figure 16.7

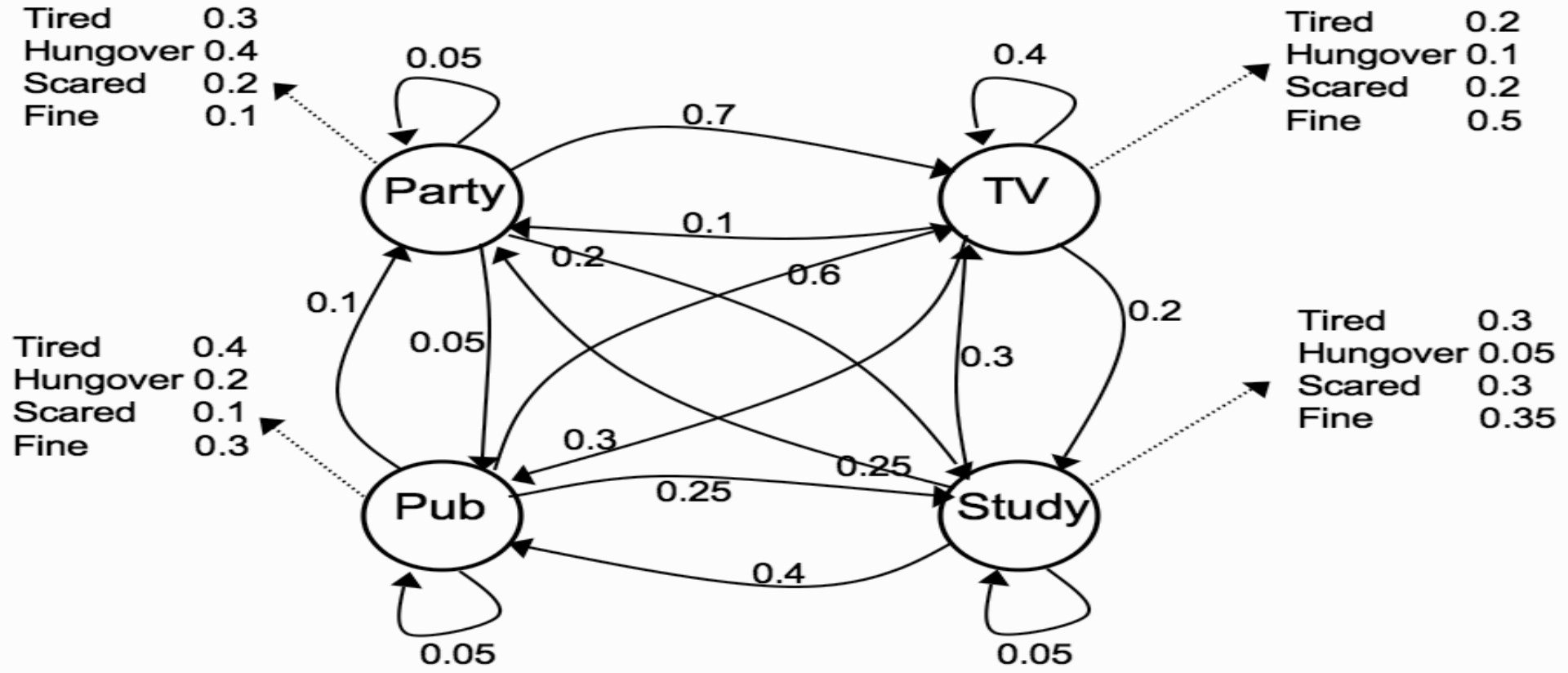


FIGURE 16.7 The example HMM with transition and observation probabilities shown.

	Previous night			
	TV	Pub	Party	Study
TV	0.4	0.6	0.7	0.3
Pub	0.3	0.05	0.05	0.4
Party	0.1	0.1	0.05	0.25
Study	0.2	0.25	0.2	0.05

	TV	Pub	Party	Study
Tired	0.2	0.4	0.3	0.3
Hungover	0.1	0.2	0.4	0.05
Scared	0.2	0.1	0.2	0.3
Fine	0.5	0.3	0.1	0.35

and then the observation probabilities:

THE FORWARD ALGORITHM

- Following are the observations: *O = (tired, tired, fine, hungover, hungover,* scared, hungover, fine) and want to work out the likely run of states that generated it.
- The probability of observations *O = {o(1), . . . , o(T)} come from the model can be* computed using simple conditional probability.
- Knowing that you were doing something last night, so for an observation *o(t) =tired (say) just need to compute the probability that made* that observation given you were in a particular state (say watching TV) and multiply it by the probability that you were in that state given the state thought you were in the night before (say partying).

- So for the example, compute the probability that you were tired given that you were watching TV, which is 0.2, and then multiply it by the probability that you spent last night watching TV given that I thought you were partying the night before, which is 0.1.
- So this yields probability 0.02 for this particular state change.
- There is one extra thing that we need which is to decide which state you actually start in and as don't know this, assign probability 0.25 to each state.
- If there are *N hidden states then there are* N^T *possible* sequences, and for each one we have to compute a product of *T Probabilities*.
- *Not only will* these probabilities be incredibly small, but the computational cost of getting them will be astronomical: O(N^T *T)*.

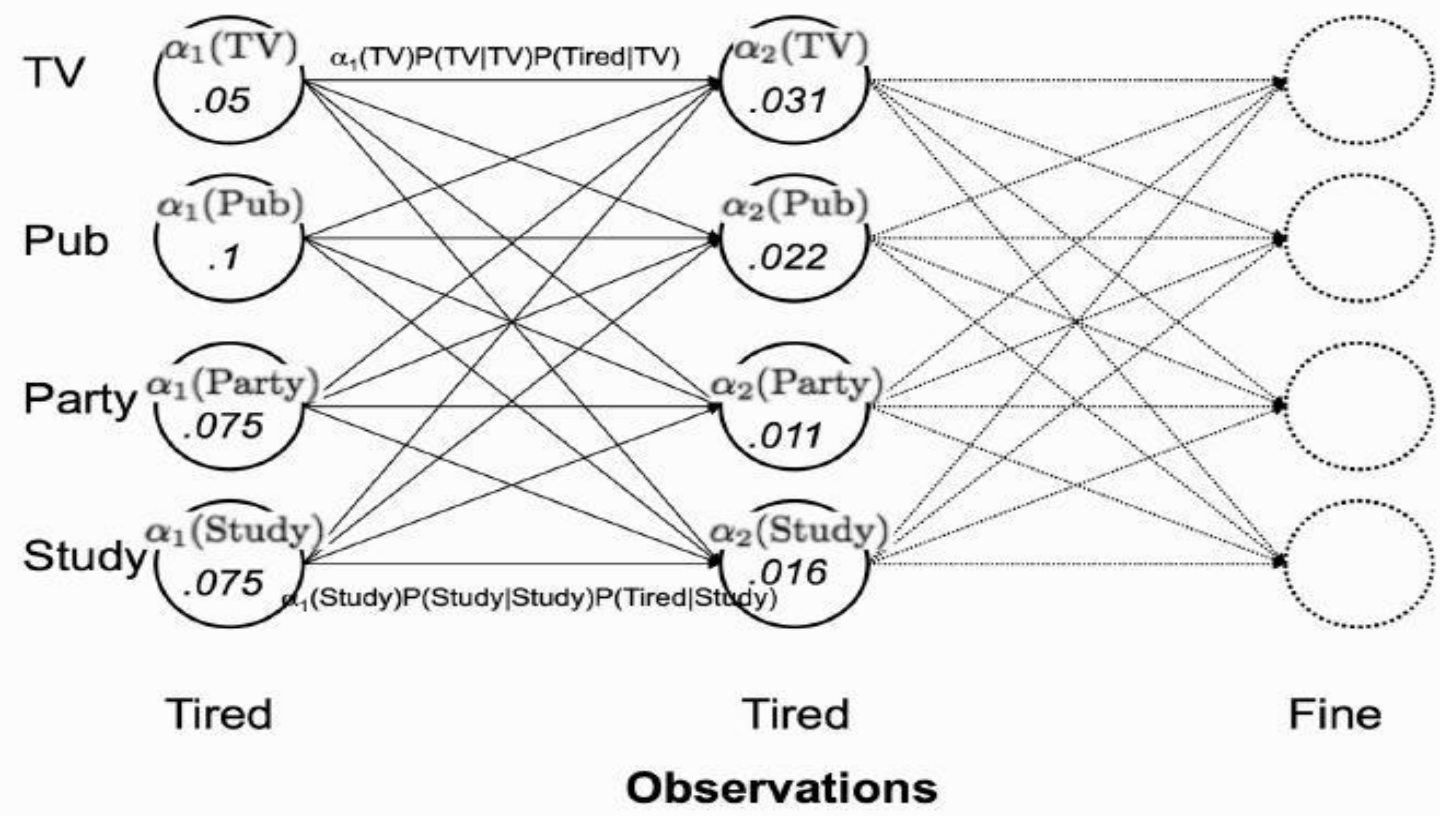


FIGURE 16.8 The forward trellis (graph with vertical slices) for the first two observations of the example HMM

- To construct the trellis we introduce a new variable $\alpha_i(t)$ *that describes the probability that at time t*
- the state is ω_i *and that the first* $(t-1)$ *steps all matched the observations o(t):*

$$\alpha_j(t) = \begin{cases} 0 & t = 0, j \neq \text{initial state} \\ 1 & t = 0, j = \text{initial state} \\ \sum_i \alpha_i(t-1) a_{i,j} b_j(o_t) & \text{otherwise.} \end{cases} \qquad (16.14)$$

- Computing *P(O) now requires only $O(N^2T)$, which is a substantial improvement.*

The HMM Forward Algorithm

- Initialise with $\alpha_{i,0} = \pi_i b_i(o_0)$
- For each observation in order $o_t,\ t = 1, \ldots, T$
  - for each of the N_s possible states s:
    - $\alpha_{s,t+1} = b_s(o_{t+1}) \left(\sum_{i=1}^{N} (\alpha_{i,t} a_{i,s}) \right)$

The Viterbi Algorithm

- The Viterbi algorithm is a dynamic programming algorithm for finding the most likely sequence of hidden states in a Hidden Markov Model (HMM)
- The next problem that we want to solve is the decoding problem of working out the hidden states: I can use my model of how students are expected to behave and match them with my observations to guess what you have been doing each evening.
- The algorithm is known as the Viterbi algorithm after its creator, although he actually derived it for error correction, a completely different application.

The HMM Viterbi Algorithm

- Start by initialising $\delta_{i,0}$ by $\pi_i bi(o_0)$ for each state i, $\phi_0 = 0$
  - run forward in time t:
    - for each possible state s:
      - $\delta_{s,t} = \max_i(\delta_{i,t-1} a_{i,s}) b_s(o_t)$
      - $\phi_{s,t} = \arg\max_i(\delta_{i,t-1} a_{i,s})$
  - set q_T^*, the most likely end hidden state to be $q_T^* = \arg\max_i \delta_{i,T}$
  - run backwards in time computing:
    - $q_{t-1}^* = \phi_{q_t^*,t}$

- Figure 16.9 shows the path for the first three states of the example.

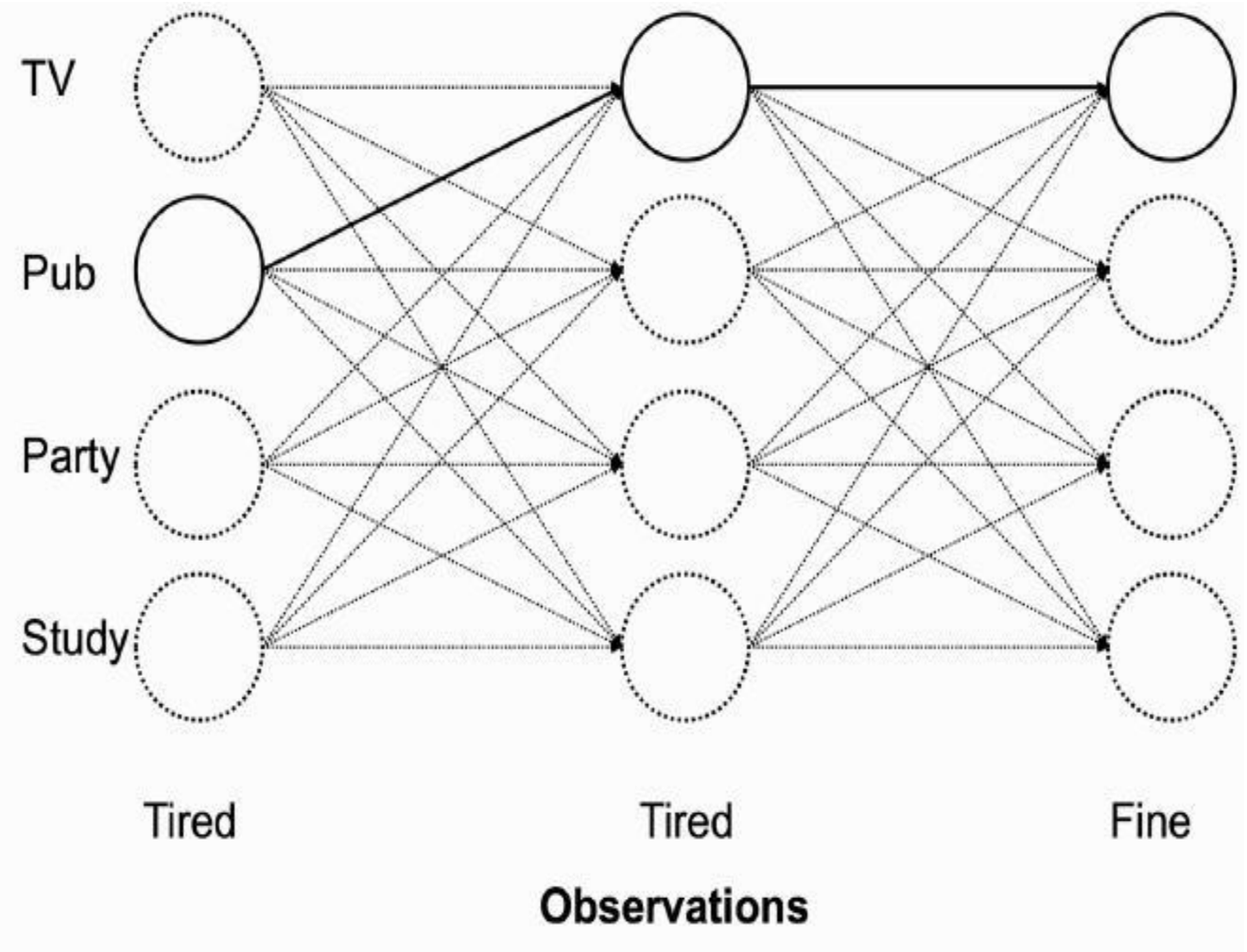


FIGURE 16.9 The Viterbi trellis for the first three observations of the example HMM.

THE BAUM–WELCH OR FORWARD–BACKWARD ALGORITHM

- Generate the HMM from sets of observations rather than by making up the transition probabilities.
- This is a learning process, and it is an unsupervised learning problem since we don’t have any target solutions to go on.
- Finding the optimal probabilities is an NP-complete problem, since we have to search over all the possible sets of probabilities for all the possible sequences.
- Instead, we will use an EM algorithm known as the Baum–Welch algorithm.

TRACKING METHODS

- Look at two methods of performing tracking.
- Perform tracking fairly easily, keeping tabs on where something is and how it is moving.
- This has an obvious evolutionary benefit, since keeping track of where predators were and whether they were coming towards you could keep you alive.
- It is also useful for a machine to be able to do this, both for similar reasons to a human or animal (watching something moving and predicting what path it will follow, for example in radar or other imaging method) and to keep track of a changing probability distribution.
- We will look at two methods of doing it, the Kalman filter and the particle filter.

THE KALMAN FILTER

- The Kalman filter (named for E. Kalman; although he was not the original inventor he did do quite a lot of work on it) is a recursive estimator.
- It makes an estimate of the next step, then computes an error term based on the value that was actually produced in the next step, and tries to correct it.
- Uses both of those to make the next prediction, and iterates this procedure.
- It is a simple cycle of predict-correct behaviour, where the error at each step is used to improve the estimate at the next iteration.
- The Kalman filter can be represented by the graphical model shown in Figure 16.10.

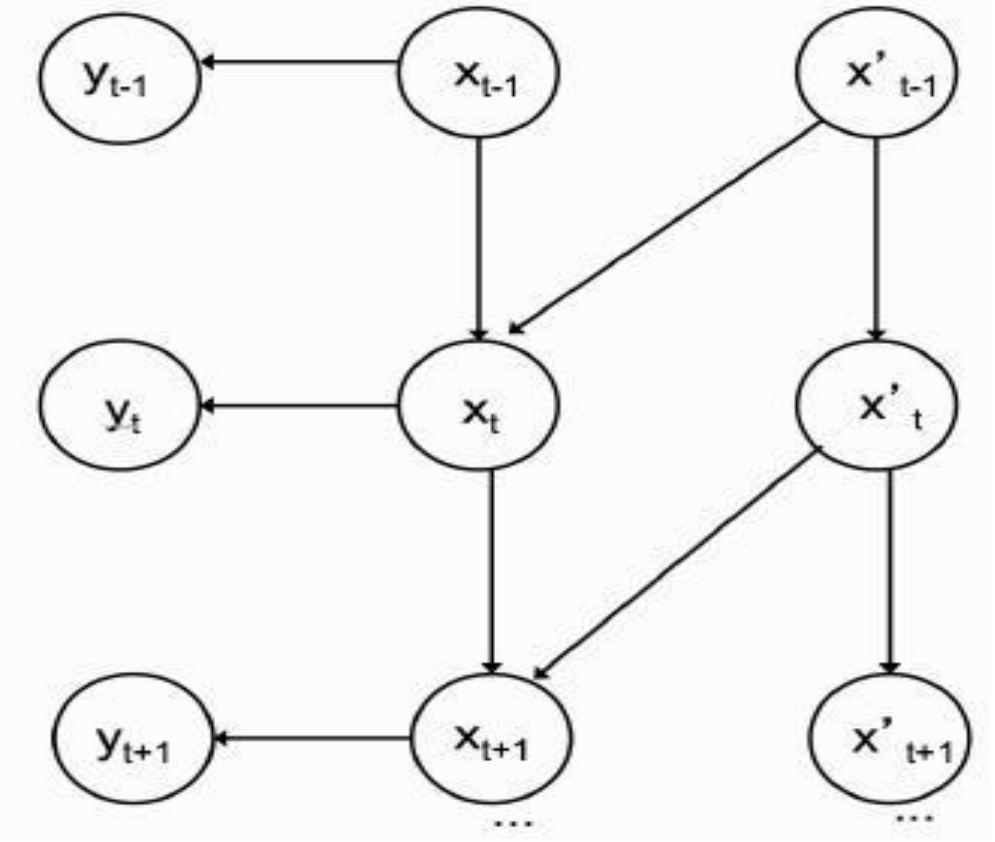


FIGURE 16.10 A representation of the Kalman filter with time derivatives (such as for tracking) as a graphical model.

- The underlying idea is that there is some time-varying process that is generating a set of noisy outputs, where there are two sources of noise.
- Write the process as a stochastic difference equation in x, which has n dimensions: $\mathbf{x}_{t+1} = \boldsymbol{A}\mathbf{x}_t + \boldsymbol{B}\mathbf{u}_t + \mathbf{w}_t$ - -- ***(16.21)***

- where A_t is an n × n matrix that represents the non-driven part of the underlying process, B is an n × l matrix that represents the driving force, and u is the l-dimensional driving force.
- w is the process noise, which is assumed to be zero mean with standard deviation Q.
- The observations that we make are m-dimensional:

$$\mathbf{y_t = Hx_t + v_t} \quad \text{---} \quad \mathbf{(16.22)}$$

- Since there is no driving force the next term is $Bu_t = 0$.
- The process equation is then derived by using Newton's laws of motion (so $x_{t+1} = x_t + \Delta t_x \cdot t$, and since we are making indexing by time, $\Delta_t = 1$, and the velocity is constant)

$$\begin{pmatrix} x_{t+1} \\ \dot{x}_{t+1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} x_t \\ \dot{x}_t \end{pmatrix} + \mathbf{w}_t. \qquad (16.23)$$

- We can observe the position of the particle, up to measurement noise, but not the velocity, and so the measurement equation is:

$$y_t = \begin{pmatrix} 1 & 0 \end{pmatrix} \begin{pmatrix} x_t \\ \dot{x}_t \end{pmatrix} + \mathbf{v}_t. \qquad (16.24)$$

- The principal simplifying assumptions of the Kalman filter are that the process is linear and that all of the distributions are Gaussian with constant covariance. Since the Convolution of Gaussians is also Gaussian, this means that we can put them together to form new Gaussians, and so the model stays well behaved.

THE PARTICLE FILTER

- As well as the linearity of the function, the Kalman filter also assumes that the distributions are Gaussian, so that they can be convolved and stay as Gaussians.
- The particular sampling technique that we will use is the sampling importance - re-sampling algorithm, which forms the basis of the particle filter, or condensation method.
- The idea is to use sampling to keep track of the state of the probability distribution.
- This is known as sequential sampling, since we are using a set of samples for time *t to estimate the process at* time *t+1, and then re-sampling from there.*

- In tracking, prior history can be useful, which means that the Markov assumption is a bad one.
- A schematic of one iteration of the particle filter is shown in Figure 16.16.

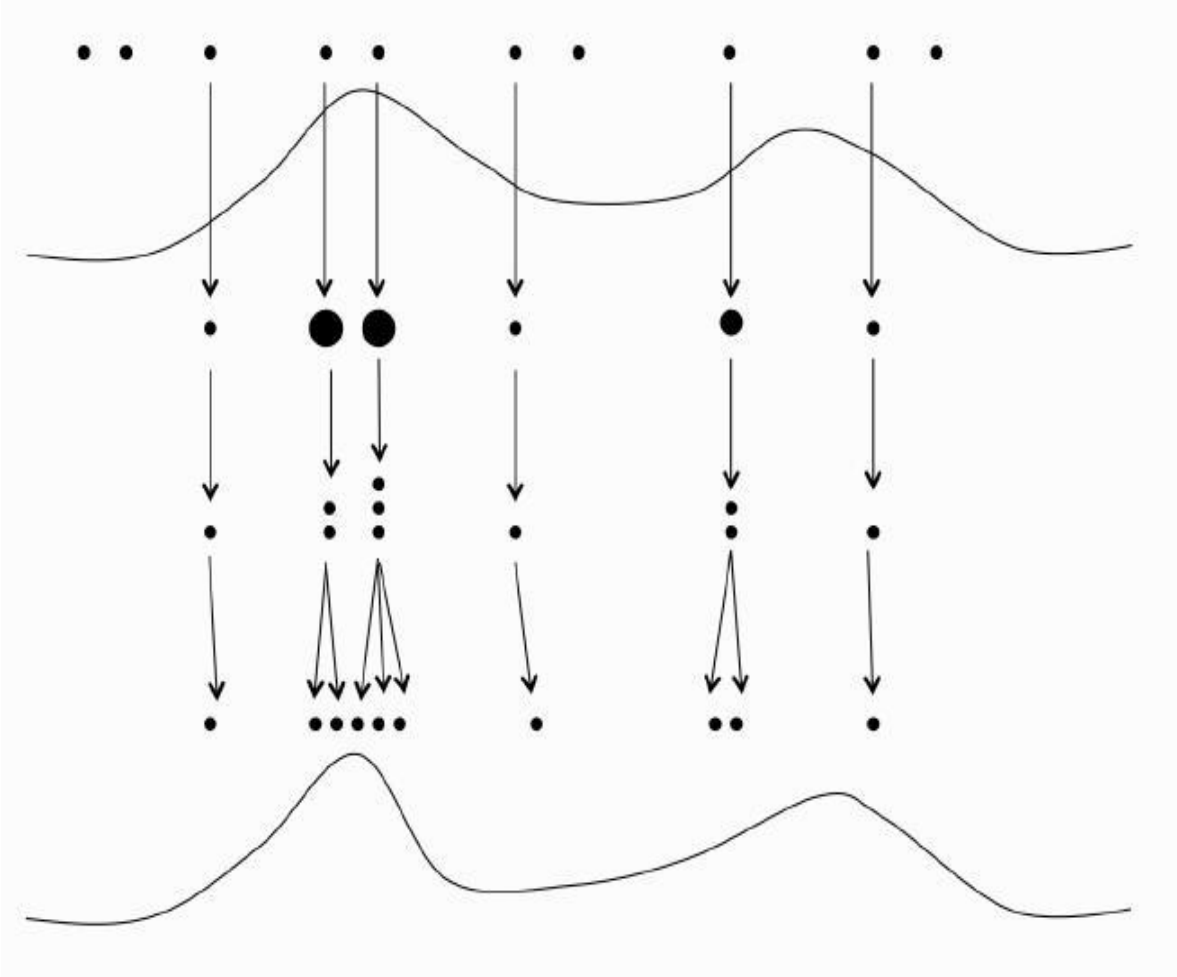


FIGURE 16.16 A schematic of one iteration of a particle filter. A set of random particle positioned have importance weights computed according to the distribution, and then new particles are created and modified based on these weights, and the estimated distribution is updated.

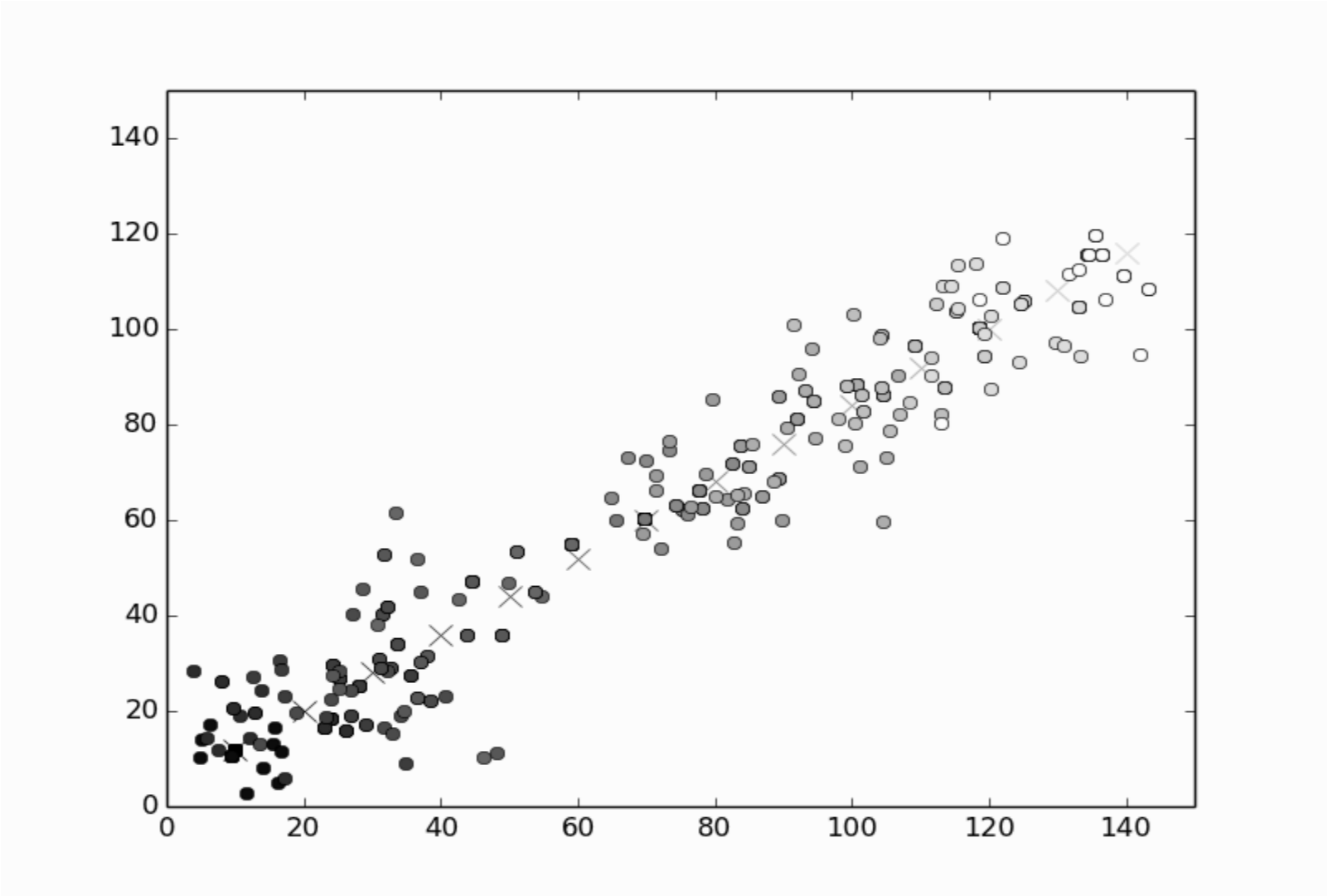


FIGURE 16.18 A particle filter tracking an object moving at a constant speed upwards and to the right in 2D. The crosses show the position of the object, and the circles show 15 particles at each iteration, starting at black (*t = 0) and fading to white (t = 10). It can* be seen that the particles track the object successfully.

THANK YOU